

# OpenG2P Gen 2 Farmer Registry

ODK Central & Ingestion Architecture: Complete Technical & Operational Reference Guide

ODK Central v2024.1

OpenG2P Gen 2

Connector Service v1.2

Celery + MinIO Jinja2

DOCUMENT VERSION

2.0.0 (Production Release)

DEPLOYED ODK SERVER

https://odk.13.207.43.8.nip.io

CONNECTOR PIPELINE ID

farmer-odk-pipeline-01 (Active)

DATE OF PUBLICATION

October 2026

ODK PROJECT & FORM

Project 13 / Form: farmer\_profile

TARGET INGEST ENDPOINT

http://farmer-registry-partner-api:8000

TABLE OF CONTENTS

1. Executive Summary

2. End-to-End System Architecture

3. Server & Service Inventory

4. ODK Form Architecture (v2)

5. Field-by-Field Mapping Matrix
6. Database Seeds & Config SQL

7. Mobile Enumerator Guide

8. Manual Testing & Verification

9. Operational Troubleshooting

10. Appendix & Useful Commands

1. Executive Summary

This document delivers the comprehensive end-to-end technical reference and operational runbook for the automated ingestion of Ethiopian smallholder farmer profiling data from **ODK Central** into the **OpenG2P Gen 2 Farmer Registry**.

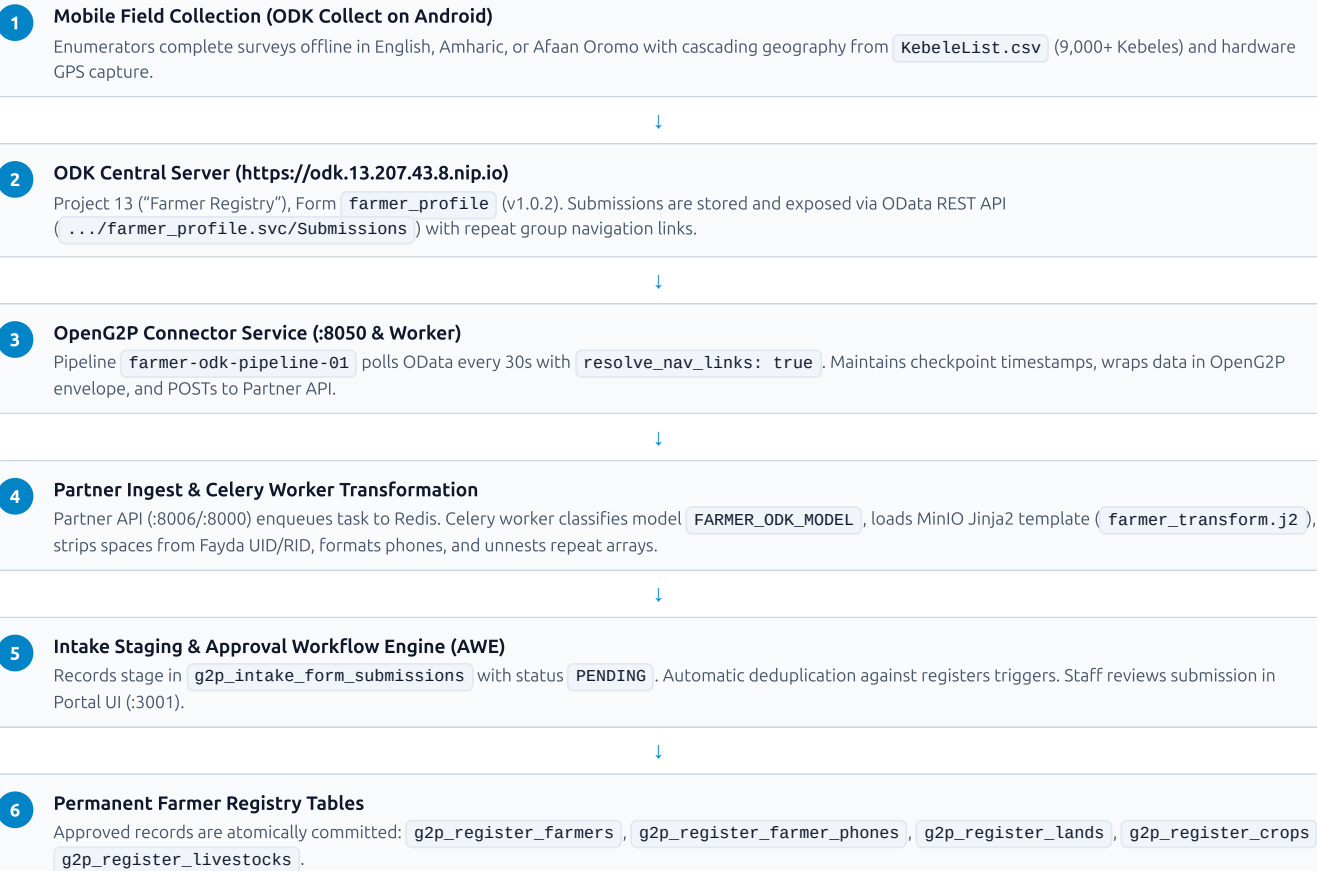
The architecture replaces Gen 1 batch manual spreadsheet imports with an automated, event-driven, decoupled integration. Offline mobile data collection by field enumerators across rural Ethiopia is automatically collected, normalized, transformed, and staged in OpenG2P with full deduplication, relational unnesting of repeat groups (land parcels, crops, livestock), and multi-lingual support.

CORE PRINCIPLES OF GEN 2 INGESTION

- Offline First:** Survey forms cache over 9,000 Kebeles and Primary Cooperatives locally on mobile devices.
- Zero-Loss Decoupling:** The OpenG2P Connector Service maintains strict cursor timestamps and idempotency keys to ensure exactly-once ingestion.
- Four-Eyes Verification:** Raw submissions first enter an Intake Staging area (AWE approval workflow) before staff review commits them to permanent registry tables.

2. End-to-End System Architecture

The data flow spans six distinct processing stages from field collection to permanent storage:



## 3. Server & Infrastructure Inventory

### 3.1 Upstream ODK Central Server Configuration

| Parameter              | Configuration Value                                               | Technical Notes                                                                           |
|------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Base Server URL        | <code>https://odk.13.207.43.8.nip.io</code>                       | Production ODK Central deployment (Nginx + OpenRosa).                                     |
| Admin Web Console      | <code>vilbertraj21@gmail.com</code> / <code>odksandbox</code>     | Full administrator rights to projects, forms, and field keys.                             |
| Project ID / Name      | <code>13</code> · <code>Farmer Registry</code>                    | Contains all farmer profile forms and media lookup tables.                                |
| Form XML ID            | <code>farmer_profile</code>                                       | Identifier matching the OData endpoint and connector settings.                            |
| Form Name / Version    | <code>Farmer Profiling Data Collection Form</code> (v1.0.2)       | Active, published version accepting submissions.                                          |
| OData Endpoint         | <code>/v1/projects/13/forms/farmer_profile.svc/Submissions</code> | Polled by the OpenG2P Connector with OData navigation expansions.                         |
| Field Key 1 (App User) | <code>Field_Officer_1</code>                                      | Token:<br><code>CBgWA1NO\$BWgO5vTjqPajzi2TLAmHU0eAf6CH9IYyGRP\$BvyotGIVLVA8bIHgseT</code> |
| Field Key 2 (App User) | <code>QA Rohit</code>                                             | Token:<br><code>u\$FxbwULP7R0gjoHx1GrZQrj70SMtv7lUtGAd6AswdavpdMqqm1A83F0\$mzI!IDf</code> |

### 3.2 Local / Platform Container Stack Inventory

| Service Name     | Docker Container                              | Host Port                             | Internal Port                         | Operational Role                                                                                 |
|------------------|-----------------------------------------------|---------------------------------------|---------------------------------------|--------------------------------------------------------------------------------------------------|
| Connector UI     | <code>farmer-registry-connector-ui</code>     | <code>5173</code>                     | <code>8080</code>                     | Management dashboard to view pipelines and ingestion runs.                                       |
| Connector API    | <code>farmer-registry-connector-api</code>    | <code>8050</code>                     | <code>8050</code>                     | FastAPI backend for connector configuration and poll triggers.                                   |
| Connector Worker | <code>farmer-registry-connector-worker</code> | N/A                                   | N/A                                   | Celery background task runner polling ODK every 30 seconds.                                      |
| Partner API      | <code>farmer-registry-partner-api</code>      | <code>8006</code>                     | <code>8000</code>                     | Secure ingress API receiving payloads via <code>/partner/ingest_data</code> .                    |
| Staff API        | <code>farmer-registry-staff-api</code>        | <code>8001</code>                     | <code>8000</code>                     | Core Registry domain REST service (FastAPI).                                                     |
| Staff Portal UI  | <code>farmer-registry-staff-ui</code>         | <code>3001</code>                     | <code>3000</code>                     | Next.js Web UI for registry officers and data approvers.                                         |
| PostgreSQL 16    | <code>farmer-registry-postgres</code>         | <code>5445</code>                     | <code>5432</code>                     | Databases: <code>farmer_registry_db</code> , <code>connector</code> , <code>master_data</code> . |
| Redis Broker     | <code>farmer-registry-redis</code>            | <code>6387</code>                     | <code>6379</code>                     | Message broker for Celery task queuing and deduplication cache.                                  |
| MinIO Storage    | <code>farmer-registry-minio</code>            | <code>9002</code> / <code>9001</code> | <code>9000</code> / <code>9001</code> | S3 object storage hosting <code>templates/farmer_transform.j2</code> .                           |
| AWE Engine       | <code>farmer-registry-awe-api</code>          | <code>8030</code>                     | <code>8000</code>                     | Approval Workflow Engine managing four-eyes approval lifecycles.                                 |

### 3.3 Deployed Server Environment Inventory (far.openg2p.test)

| Server Component          | URL / Internal Endpoint                                                    | Operational Role                                                                        |
|---------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| ODK Central Server        | <code>https://odk.13.207.43.8.nip.io</code>                                | Upstream cloud data collection server (Project 13, Form: <code>farmer_profile</code> ). |
| Server Connector Admin UI | <code>https://connector-farmer-registry.far.openg2p.test</code>            | Server dashboard for monitoring pipelines, runs, and Dead Letter Queue.                 |
| Server Connector API      | <code>https://connector-farmer-registry.far.openg2p.test/connectors</code> | Server REST API managing pipelines and manual poll triggers.                            |

| Server Component             | URL / Internal Endpoint                                                      | Operational Role                                                   |
|------------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------|
| Server Active Pipeline ID    | 9afff094f9a24a689fd53ce46af32cc4                                             | Live server pipeline polling ODK Central every 30s.                |
| Server Staff Portal UI       | https://farmer-registry.far.openg2p.test                                     | Server Web UI for reviewing & approving farmer intake submissions. |
| Server Partner Ingest Target | http://farmer-registry-partner-api.far.svc.cluster.local/partner/ingest_data | Internal Kubernetes cluster DNS target used by Connector Worker.   |

### 3.4 Data Routing & Architecture FAQs: Localhost vs. Remote Server

#### FAQ 1: Why did my colleague's submission from another state appear in my localhost:3001?

**The Mechanism:** ODK Central ( `https://odk.13.207.43.8.nip.io` ) is a single, centralized cloud server hosted on AWS.

**The Flow:** When a colleague in another state fills out the form in ODK Collect and submits it, the data reaches the shared cloud ODK Central server. If your local Docker stack is running on your laptop, its background container `farmer-registry-connector-worker` polls the cloud server every 30 seconds, fetches all new submissions across the internet, and stores them in your local PostgreSQL database ( `localhost:5445` ). Thus, both your test and your colleague's test appear in your local Staff Portal ( `localhost:3001` ).

*Conclusion:* This proves the end-to-end ingestion pipeline, Jinja2 transformation, and intake staging workflow are 100% operational.

#### FAQ 2: What happens if the Connector on the server uses localhost instead of the cluster URL?

**In Simple Terms:** In containerized environments (Docker and Kubernetes), `localhost` means "inside this pod/container only".

If the connector worker pod on the server sends data to `http://localhost:8000`, it is looking for the Partner API inside itself, where nothing is listening, resulting in `Connection Refused`. On the remote server, it MUST use the Kubernetes service DNS: `http://farmer-registry-partner-api.far.svc.cluster.local/partner/ingest_data`.

### 3.5 Remote Server Deployment Status & DevOps Action Checklist

The pipeline has been created and verified on the server:

- **Server Connector Admin UI:** `https://connector-farmer-registry.far.openg2p.test`
- **Configured Pipeline ID:** `9afff094f9a24a689fd53ce46af32cc4`
- **Status:** Active & Polling (fetched 16 submissions from ODK Central).
- **Current Server Blocker:** Partner API returns `HTTP 500 Internal Server Error` because database seeds are missing on the server.

#### Mandatory DevOps Action Items on Server:

1. In **master\_data DB**: `INSERT INTO public.g2p_partners (partner_id, partner_mnemonic, keymanager_reference_id, is_active) VALUES ('farmer-partner', 'Farmer', 'farmer-key-ref', true) ON CONFLICT (partner_id) DO NOTHING;`
2. In **farmer\_registry\_db DB**: Run data model and key path seeds for `FARMER_ODK_MODEL`.
3. In **Server MinIO**: Upload `odk/templates/farmer_transform.j2` into the `templates` bucket.
4. **Verify:** Click Replay in the Server Connector UI and verify intake records appear in `https://farmer-registry.far.openg2p.test/en/intake-form/farmer`.

## 4. ODK Form Design & XLSForm Specifications

The profiling form ( `odk/ATI_Farmers_Profile_ODK_Form_v2.xlsx` ) is structured into 14 logical sections, translated into three languages, and integrated with offline lookup datasets.

### 4.1 Tri-Lingual Language Architecture

1. **English ( `en` )**: Default base language used for system labels, enumerator instructions, and fallback text.
2. **Amharic ( `am` ) — አማርኛ**: Official federal working language; native Ethiopic Ge'ez script stored directly in PostgreSQL UTF-8 columns.
3. **Afaan Oromo ( `om` ) — Afaan Oromoo**: Widely spoken regional language; stored in designated language-specific profile columns.

### 4.2 Preloaded Media Attachments

- **KebeleList.csv** : Over 9,000 Ethiopian Kebeles pre-indexed with hierarchical codes ( `region_code` , `zone_code` , `woreda_code` ). Uses ODK's `select_one_from_file` with dynamic `choice_filter` for instant cascading drop-downs without memory lag.
- **PrimaryCoopList.csv** : Preloaded directory of Primary Agricultural Cooperatives and Unions.

### 4.3 Complete Questionnaire Section Breakdown

| Section Identifier                   | Key Question Fields                                                                                                                                                                                                        | Technical Characteristics                                                                |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| 1. <code>consent_form_section</code> | <code>consent_form</code> (agree / disagree)                                                                                                                                                                               | Mandatory gatekeeper. Rejecting exits survey immediately.                                |
| 2. <code>household_questions</code>  | <code>household_head</code> , <code>head_registered</code> , <code>relationship_to_head</code>                                                                                                                             | Identifies if respondent is the family head or dependent.                                |
| 3. <code>national_id_section</code>  | <code>national_id</code> , <code>national_uid</code> , <code>national_rid</code> , <code>other_id</code>                                                                                                                   | Captures Fayda National ID (16 digits) and RID (29 digits).                              |
| 4. <code>locale_info</code>          | <code>region</code> , <code>zone</code> , <code>woreda</code> , <code>kebele</code> , <code>language</code>                                                                                                                | Cascading administrative boundaries linked to KebeleList.csv.                            |
| 5. <code>personal_info</code>        | 3-part names (EN, AM, OM), <code>gender</code> , <code>dob_gc</code> , <code>dob_ec</code> , <code>primary_phone_number</code> , <code>secondary_phone_number</code> , <code>farming_type</code> , <code>disability</code> | Supports Gregorian and Ethiopian calendar dates. Normalizes phone numbers automatically. |
| 6. <code>socio_economic_data</code>  | <code>marital_status</code> , <code>education_level</code> , <code>income_source</code>                                                                                                                                    | Captures social vulnerability and livelihood classification.                             |
| 7. <code>membership</code>           | <code>primary_cooperative</code> , <code>coop_union</code> , <code>farmer_cluster</code> , <code>primary_commodity</code>                                                                                                  | Links smallholder to organized agricultural supply chains.                               |
| 8. <code>land_info_repeat</code>     | <code>land_ownership</code> , <code>total_land_area</code> , <code>land_id</code> , <code>land_certificate</code> (photo attachment)                                                                                       | <b>Repeat group</b> allowing multiple land parcels per farmer.                           |
| 9. <code>crop_information</code>     | <code>crop_name_rep</code> , <code>crop_date</code> , <code>crop_water_source</code>                                                                                                                                       | <b>Repeat group</b> capturing planted crops and sowing seasons.                          |
| 10. <code>livestock_info</code>      | <code>animal_rep</code> , <code>num_animals</code> , <code>livestock_water_source</code>                                                                                                                                   | <b>Repeat group</b> capturing cattle, sheep, goats, poultry.                             |
| 11. <code>agricultural_input</code>  | Fertilizer, pesticide, insecticide, improved seed use                                                                                                                                                                      | Boolean flags and quantities utilized per season.                                        |
| 12. <code>access_to_resource</code>  | Machinery access (Tractor, Harvester), Bank loans, Savings                                                                                                                                                                 | Captures mechanization and financial inclusion metrics.                                  |
| 13. <code>other_hh_members</code>    | Name, relationship, gender, DOB for each member                                                                                                                                                                            | <b>Repeat group</b> registering secondary family members.                                |
| 14. <code>farmer_location</code>     | <code>location</code> (geopoint string: lat lon alt acc)                                                                                                                                                                   | High-precision hardware GPS coordinates of the farmstead.                                |

## 5. Field-by-Field Mapping Matrix

The Registry Celery Worker uses the MinIO template `farmer_transform.j2` to render incoming ODK Central OData payloads into the canonical OpenG2P Intake Schema.

| ODK Question JSON Path                              | Target Table                                  | Target Column                                                                       | Data Type | Transformation & Sanitization Rule                                                                                |
|-----------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------|-----------|-------------------------------------------------------------------------------------------------------------------|
| <code>personal_info/first_name_english</code>       | <code>g2p_register_farmers</code>             | <code>first_name</code>                                                             | VARCHAR   | Trims whitespace; Latin given name.                                                                               |
| <code>personal_info/father_name_english</code>      | <code>g2p_register_farmers</code>             | <code>middle_name</code>                                                            | VARCHAR   | Father's name (Ethiopian naming patronymic).                                                                      |
| <code>personal_info/grandfather_name_english</code> | <code>g2p_register_farmers</code>             | <code>last_name</code>                                                              | VARCHAR   | Grandfather's name (Family name).                                                                                 |
| <code>personal_info/first_name_amharic</code>       | <code>g2p_register_farmers</code>             | <code>first_name_amh</code>                                                         | VARCHAR   | Native Amharic Ge'ez text preserved in UTF-8.                                                                     |
| <code>personal_info/father_name_amharic</code>      | <code>g2p_register_farmers</code>             | <code>middle_name_amh</code>                                                        | VARCHAR   | Amharic father's name.                                                                                            |
| <code>personal_info/grandfather_name_amharic</code> | <code>g2p_register_farmers</code>             | <code>last_name_amh</code>                                                          | VARCHAR   | Amharic grandfather's name.                                                                                       |
| <code>personal_info/first_name_other</code>         | <code>g2p_register_farmers</code>             | <code>first_name_om</code>                                                          | VARCHAR   | Afaan Oromo given name.                                                                                           |
| <code>personal_info/gender</code>                   | <code>g2p_register_farmers</code>             | <code>gender</code>                                                                 | ENUM      | Normalized to uppercase: <code>MALE</code> , <code>FEMALE</code> , or <code>UNKNOWN</code> .                      |
| <code>personal_info/date_of_birth</code>            | <code>g2p_register_farmers</code>             | <code>birth_date</code>                                                             | DATE      | Gregorian date parsed to <code>YYYY-MM-DD</code> .                                                                |
| <code>personal_info/date_of_birth_ec</code>         | <code>g2p_register_farmers</code>             | <code>birth_date_ec</code>                                                          | DATE      | Ethiopian calendar string preserved as <code>YYYY-MM-DD</code> .                                                  |
| <code>household_questions/household_head</code>     | <code>g2p_register_farmers</code>             | <code>is_household_head</code>                                                      | BOOLEAN   | Evaluated as <code>true</code> if value in <code>['yes', '1', 'true']</code> .                                    |
| <code>national_id_section/national_uid</code>       | <code>g2p_register_farmer_id_documents</code> | <code>value</code> (UID)                                                            | VARCHAR   | <b>Space Stripping:</b><br><code>replace(' ', '')</code> turns <code>xxxx xxxx...</code> into 16 digits.          |
| <code>national_id_section/national_rid</code>       | <code>g2p_register_farmer_id_documents</code> | <code>value</code> (RID)                                                            | VARCHAR   | 29-digit Fayda Registration ID; stripped of all spaces.                                                           |
| <code>farmer_reference_id_section/...</code>        | <code>g2p_register_farmer_id_documents</code> | <code>value</code> (ODK_ACK)                                                        | VARCHAR   | Unique ODK Acknowledgement token.                                                                                 |
| <code>personal_info/primary_phone_number</code>     | <code>g2p_register_farmer_phones</code>       | <code>phone_number</code>                                                           | VARCHAR   | Strips <code>+251 / 251</code> , hyphens, and spaces; tagged <code>PRIMARY</code> .                               |
| <code>personal_info/secondary_phone_number</code>   | <code>g2p_register_farmer_phones</code>       | <code>phone_number</code>                                                           | VARCHAR   | Normalizes optional backup phone; tagged <code>SECONDARY</code> .                                                 |
| <code>farmer_location/location</code>               | <code>g2p_register_farmers</code>             | <code>enumerator_latitude</code><br><code>longitude</code><br><code>altitude</code> | FLOAT     | Splits space-delimited geopoint: <code>parts[0]</code> Lat, <code>parts[1]</code> Lon, <code>parts[2]</code> Alt. |
| <code>land_info/land_info_repeat</code>             | <code>g2p_register_lands</code>               | Multiple                                                                            | ARRAY     | Unnests into <code>land_id</code> , <code>area_in_hectare</code> , <code>land_ownership_type</code> .             |
| <code>crop_information/crop_repeat</code>           | <code>g2p_register_crops</code>               | Multiple                                                                            | ARRAY     | Unnests into <code>commodity</code> , <code>planted_date</code> , <code>season</code> .                           |

| ODK Question JSON Path          | Target Table            | Target Column | Data Type | Transformation & Sanitization Rule                            |
|---------------------------------|-------------------------|---------------|-----------|---------------------------------------------------------------|
| livestock_info/livestock_repeat | g2p_register_livestocks | Multiple      | ARRAY     | Unnests into livestock_type , head_count , livestock_system . |

## 6. Database Seeds & Configuration SQL

The ingestion pipeline requires initial seed records across three databases: `master_data`, `farmer_registry_db`, and `connector`.

### 6.1 Register Ingest Partner in `master_data` Database

```
-- Target Database: master_data
INSERT INTO public.g2p_partners (
  partner_id, partner_mnemonic, keymanager_reference_id, is_active
) VALUES (
  'farmer-partner', 'Farmer', 'farmer-key-ref', true
) ON CONFLICT (partner_id) DO NOTHING;
```

### 6.2 Register Data Model, Key Paths & Semantic Patterns in `farmer_registry_db` Database

```
-- Target Database: farmer_registry_db
-- 1. Register Data Model
INSERT INTO public.data_models (
  data_model_id, data_model_mnemonic, pattern_for_data_model, response_template_document_id, is_active
) VALUES (
  'FARMER_ODK_MODEL', 'FARMER_ODK_MODEL', '$.body.header.sender_id=>^.*$', '1f0953d4-f0fb-4336-a126-4d0519a74ffc', true
) ON CONFLICT (data_model_id) DO UPDATE SET
  pattern_for_data_model = EXCLUDED.pattern_for_data_model,
  response_template_document_id = EXCLUDED.response_template_document_id,
  is_active = true;

-- 2. Register Incoming Key Paths
INSERT INTO public.incoming_model_key_paths (
  key_path_id, data_model_id, key_path_for_message_id, key_path_for_sender,
  key_path_for_signature, key_path_for_signature_payload, is_list, key_path_for_list_elements
) VALUES (
  'farmer_key_path', 'FARMER_ODK_MODEL',
  '$.body.header.message_id', '$.body.header.sender_id', '$.body.header.signature',
  '$.body.message', false, ''
) ON CONFLICT (key_path_id) DO UPDATE SET
  key_path_for_list_elements = EXCLUDED.key_path_for_list_elements;

-- 3. Register Incoming Model Semantic Pattern (Business Payload Mapping)
INSERT INTO public.incoming_model_semantic_patterns (
  semantic_pattern_id, data_model_id, register_id, intake_form_id, section_id,
  pattern_for_register, pattern_for_intake_form, pattern_for_section,
  key_path_for_business_payload, raw_payload_enricher_class
) VALUES (
  'farmer_odk_semantic_pattern', 'FARMER_ODK_MODEL',
  'a1a4d25a-1cd4-4356-abac-985a0b3c6bcd', 'a1a4d25a-1cd4-4356-abac-8782382649',
  NULL, NULL, '$.body.header.sender_id=>^.*$', NULL,
  '$.body.message.payload', 'G2PDciFarmerCreateEnricherService'
) ON CONFLICT (semantic_pattern_id) DO UPDATE SET
  key_path_for_business_payload = EXCLUDED.key_path_for_business_payload;
```

### 6.3 Register Pipeline Definition in `connector` Database

```
-- Target Database: connector
INSERT INTO connector_definitions (
  connector_id, name, platform, transport_type, enabled, paused,
  data_model_mnemonic, g2p_sender_id, g2p_register_mnemonic,
  source_config_json, auth_type, auth_secret_json, webhook_verifier
) VALUES (
  'farmer-odk-pipeline-01',
  'Farmer Registry - ODK Central Ingestion',
  'odk_central',
  'odk_central',
  true,
  false,
  'FARMER_ODK_MODEL',
  'farmer-partner',
  'Farmer',
  '{
    "base_url": "https://odk.13.207.43.8.nip.io",
    "project_id": 13,
    "form_id": "farmer_profile",
    "resolve_nav_links": true,
    "strict_incremental": false,
    "poll_interval_seconds": 30,
    "target_url": "http://farmer-registry-partner-api:8000/partner/ingest_data",
    "target_headers": {
      "partner-id": "farmer-partner",
      "Content-Type": "application/json"
    }
  }',
  'odk_session',
  '{
    "email": "vilbertraj21@gmail.com",
    "password": "odksandbox"
  }',
  'hmac_sha256'
)
ON CONFLICT (connector_id) DO UPDATE SET
  source_config_json = EXCLUDED.source_config_json,
  auth_secret_json = EXCLUDED.auth_secret_json,
  enabled = EXCLUDED.enabled;
```

### 6.4 Deploy Jinja2 Template to MinIO Object Storage

```
# 1. Alias the local MinIO instance (API Port: 9002)
mc alias set myminio http://localhost:9002 admin adminsecret

# 2. Upload template to templates bucket
mc cp odk/templates/farmer_transform.j2 myminio/templates/farmer_transform.j2

# 3. Verify upload
mc ls myminio/templates
```

## 7. Step-by-Step Manual Testing & Verification Protocol

Follow these three standardized methods to test the ODK Central to OpenG2P pipeline end-to-end.

### Method A: 1-Click Automated Python Test Script (Fastest)

A dedicated verification script is located at `odk/test_odk_central_submission.py`. It constructs a valid OpenRosa XML payload, submits it to ODK Central, triggers the connector poller, and checks ingestion:

```
# Execute from workspace root
python3 odk/test_odk_central_submission.py
```

#### SAMPLE OUTPUT OF A SUCCESSFUL RUN

```
[*] Submitting test record to ODK Central: https://odk.13.207.43.8.nip.io/v1/key/.../submission
[*] Farmer Name: TestManual ODKDevOps Check
[+] ODK Central accepted submission! HTTP 201 Created
[*] Triggering Connector Service poll...
[+] Triggered immediate poll on connector: task_id=8073a92d-7b1d-4689-93f6-dc45b1bde180
[*] Connector last poll status: SUCCESS
```

### Method B: Manual cURL OpenRosa Submission via Terminal

If Python is unavailable, you can submit raw XML using cURL with the `X-OpenRosa-Version: 1.0` header:

```
# 1. Post XML survey instance to ODK Central App User endpoint
curl -k -i -X POST \
  -H "X-OpenRosa-Version: 1.0" \
  -F "xml_submission_file=@/tmp/test_submission.xml" \
  "https://odk.13.207.43.8.nip.io/v1/key/CBgWA1N0\Bwg05vTjqPajzi2TlAmHU0eAf6CH9IYygRP\BvyotGIVlVA8bIHgseT/projects/13/submission"

# 2. Trigger immediate poll on Connector
curl -s -X POST http://localhost:8050/connectors/farmer-odk-pipeline-01/poll
```

### Method C: Real Mobile Field Test via ODK Collect

1. Install **ODK Collect** from Google Play Store on an Android device.
2. Log into ODK Central Web Console: <https://odk.13.207.43.8.nip.io>.
3. Navigate to **Project 13 ("Farmer Registry")** → **App Users** → Click **QR Code** next to `Field_Officer_1`.
4. Open ODK Collect on phone → Select **Configure with QR code** → Scan the screen.
5. Tap **Get Blank Form** → Download **Farmer Profiling Data Collection Form**.
6. Tap **Fill Blank Form** → Fill in farmer details → Tap **Save Form and Exit** → **Send Finalized Form**.
7. Within 30 seconds, the Connector Worker pulls the submission from ODK Central into OpenG2P.

## 8. Verification & Review Workflows

### 8.1 Connector UI Run Audit

Open <http://localhost:5173> in your browser:

- Navigate to **Pipelines** → `farmer-odk-pipeline-01`.
- Click the **Runs** tab. Verify that the recent run indicates **SUCCESS** with `attempt_count: 1`.

### 8.2 Database Verification Queries

```
# 1. Check Celery Classification & Transformation Status
docker exec -i farmer-registry-postgres psql -U postgres -d farmer_registry_db -c "
SELECT ingest_id, data_model_id, transformation_status, ingestion_status, intake_form_submission_id, classified_date_time
FROM incoming_classified_data
ORDER BY classified_date_time DESC LIMIT 1;
"

# 2. Check Staged Intake Form Submission
docker exec -i farmer-registry-postgres psql -U postgres -d farmer_registry_db -c "
SELECT submission_id, application_reference, approval_status, first_created_at
FROM g2p_intake_form_submissions
ORDER BY first_created_at DESC LIMIT 1;
"

# 3. Check Staged Farmer Personal Information
docker exec -i farmer-registry-postgres psql -U postgres -d farmer_registry_db -c "
SELECT first_name, middle_name, last_name, first_name_amh, gender, birth_date, region_name, woreda_name
FROM g2p_intake_form_farmers
ORDER BY created_at DESC LIMIT 1;
"

# 4. Check Staged Land Parcel Repeat Group
docker exec -i farmer-registry-postgres psql -U postgres -d farmer_registry_db -c "
SELECT land_id, land_ownership_type, area_in_hectare
FROM g2p_intake_form_lands
ORDER BY created_at DESC LIMIT 1;
"
```

### 8.3 Staff Review & Final Approval

1. Log in to the Staff Portal UI at <http://localhost:3001> (Credentials: `staff` / `staff`).
2. Navigate to **Intake Forms** → **Farmer Ingestion Intake**.
3. Click on the newly staged submission (e.g. `20260CT05-316599`).
4. Review the 8 tabs: Personal Details, Administrative Hierarchy, GPS Coordinates, Land Parcels, Crops, Livestock, Inputs, and Cooperatives.
5. Click **Approve**. The Approval Workflow Engine commits the submission into `g2p_register_farmers` with an assigned Functional ID (e.g. `ET-FRM-00123`).

## 9. Troubleshooting & Operational Runbook

| Issue / Error Symptom                                                                                | Root Cause                                                                                                     | Corrective Action                                                                                                                               |
|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| HTTP 400: An expected header field (X-OpenRosa-Version) did not match                                | ODK Central OpenRosa submission endpoint strictly enforces protocol versioning.                                | Pass <code>-H "X-OpenRosa-Version: 1.0"</code> in the HTTP POST request.                                                                        |
| Server error 500: Server error for url 'http://farmer-registry-partner-api:8000/partner/ingest_data' | <code>farmer-partner</code> is missing in <code>master_data.g2p_partners</code> or Partner API is unreachable. | Run the partner seed SQL in <code>master_data</code> . Inspect container logs: <code>docker logs farmer-registry-partner-api</code> .           |
| FARMER_ODK_MODEL transformation failed in Celery                                                     | MinIO bucket is missing <code>farmer_transform.j2</code> or S3 credentials failed.                             | Execute <code>mc cp odk/templates/farmer_transform.j2 myminio/templates/farmer_transform.j2</code> . Check MinIO health at <code>:9002</code> . |
| Repeat groups (Land / Crops) are missing in Intake Form                                              | <code>resolve_nav_links</code> is set to <code>false</code> in the connector source config.                    | Set <code>"resolve_nav_links": true</code> in <code>source_config.json</code> inside <code>connector.connector_definitions</code> .             |
| Connector stops polling or skips recent submissions                                                  | Cursor boundary is stuck or corrupted in <code>poll_state.json</code> .                                        | Reset cursor via API: <code>curl -X POST http://localhost:8050/connectors/farmer-odk-pipeline-01/reset-cursor</code> or clear idempotency keys. |
| All connection attempts failed in Connector Worker                                                   | Internal Docker network cannot resolve external DNS for <code>odk.13.207.43.8.nip.io</code> .                  | Test network egress: <code>docker exec -it farmer-registry-connector-worker curl -k -I https://odk.13.207.43.8.nip.io</code> .                  |
| Fayda UID validation regex failure                                                                   | Enumerator entered Fayda UID with spaces (e.g. <code>1234 5678 9012 3456</code> ).                             | Sanitized by <code>farmer_transform.j2</code> via <code>replace(' ', '')</code> . Ensure latest template is active in MinIO.                    |

## 10. Appendix: Cheat Sheet & Essential Endpoints

```
# Check connector health
curl -s http://localhost:8050/health/ready

# List active connectors
curl -s http://localhost:8050/connectors | jq

# Trigger immediate poll
curl -s -X POST http://localhost:8050/connectors/farmer-odk-pipeline-01/poll

# View recent runs
curl -s http://localhost:8050/runs | jq '.[0:3]'

# Check Celery Worker logs
docker logs --tail 50 -f farmer-registry-celery-worker

# Check Connector Worker logs
docker logs --tail 50 -f farmer-registry-connector-worker
```

**Document Approval & Maintenance:** This guide must be reviewed and re-validated upon any changes to XLSForm versions, OpenG2P core schema migrations, or ODK Central infrastructure updates.